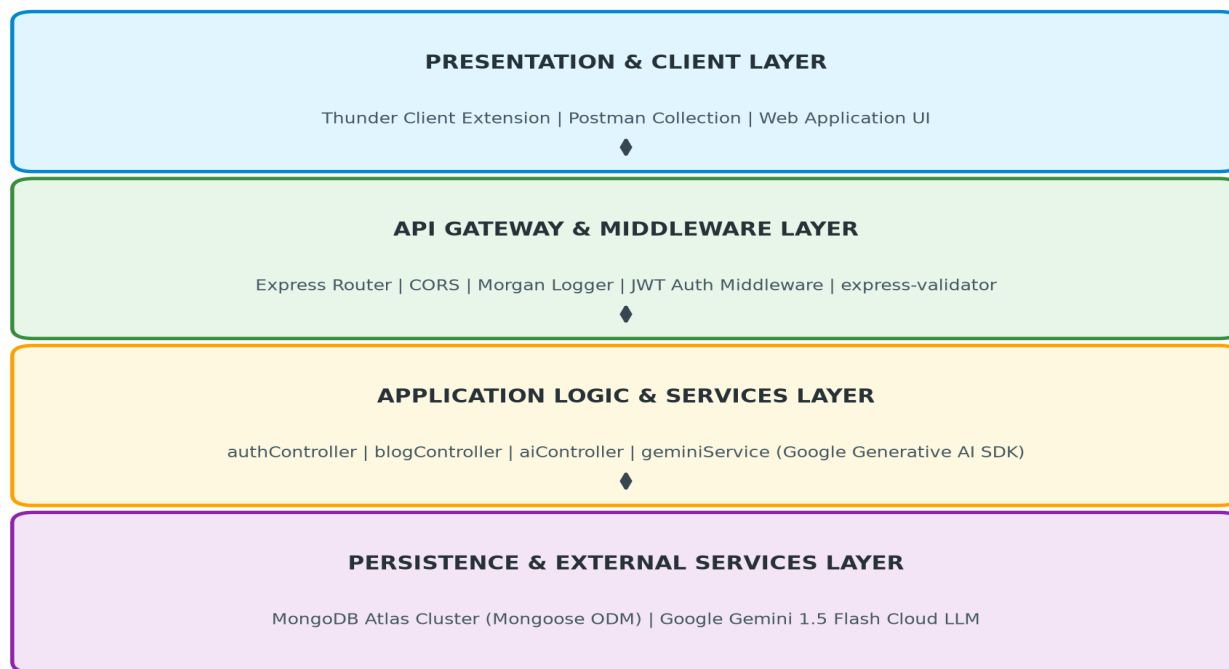


## Solution Architecture Diagram

|               |                 |                |                   |
|---------------|-----------------|----------------|-------------------|
| Date:         | 3 October 2026  | Team ID:       | UNM1611SHARVESHAM |
| Project Name: | AI BlogNest API | Maximum Marks: | 5 Marks           |

### Solution Architecture Blueprint:

AI BlogNest API is designed using a multi-layered, decoupled architecture separating presentation, gateway routing, business logic, security middleware, and external data services.



### Architectural Highlights:

- **Stateless REST Design:** Every request is self-contained with JWT Bearer authentication.
- **Decoupled AI Engine:** geminiService isolates LLM communication from HTTP route handlers.
- **Centralized Resilience:** Single error middleware catches synchronous and asynchronous failures.

## Solution Architecture — Component Description Table

### Architectural Layer Component Specifications:

| Component Name                    | Layer             | Role & Responsibilities                                                                                                                  | Technologies Used                                                       |
|-----------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>Presentation Client</b>        | Presentation      | Sends REST HTTP JSON requests (Register, Login, Generate, CRUD). Manages test collections and bearer authorization tokens.               | Thunder Client, Postman v2.1, Browser HTTP Client                       |
| <b>API Gateway &amp; Server</b>   | Gateway           | Initializes Express application, mounts CORS policy, JSON body parser, Morgan HTTP logging, and environment variable configuration.      | Node.js, Express.js (app.js, server.js), cors, morgan, dotenv           |
| <b>Security &amp; Validation</b>  | Middleware        | Enforces Bearer token extraction and verification; rejects unauthenticated requests with 401; validates request schemas with 422 errors. | jsonwebtoken, bcryptjs, express-validator, authMiddleware.js            |
| <b>Controllers &amp; Services</b> | Application Logic | Implements business logic for auth, blog operations, prompt building, regex text cleaning, and domain category classification.           | authController.js, blogController.js, aiController.js, geminiService.js |
| <b>External AI Service</b>        | External AI       | Executes generative LLM queries to produce 100-150 word blog posts and concise executive text summaries.                                 | Google Gemini API, gemini-1.5-flash, @google/generative-ai              |
| <b>Data Storage Layer</b>         | Persistence       | Stores and indexes Users and Blogs collections. Enforces schema constraints, default timestamps, and author ObjectId references.         | MongoDB Atlas, Mongoose ODM (User.js, Blog.js, db.js)                   |

### Security Boundaries & Fault Tolerance:

- **Credential Isolation:** MongoDB connection strings and Google Gemini API keys reside exclusively in .env and are never hardcoded or committed to git.
- **Resource Isolation:** A user cannot modify or delete articles authored by other users (returns 403 Forbidden).
- **Graceful Fallback:** If GEMINI\_MODEL environment is misconfigured, geminiService automatically falls back to gemini-1.5-flash.